Study Project – Phase 2 Document

Project Title: SmartCloud Secrets Manager
Team Members:

Nymish Kashivishwanath	2023EBCS686
Shrinibas Mahanta	2023EBCS695
Prem Bhavesh Shah	2023EBCS615
Abhinav Nagar	2023EBCS623

Project Advisor / Supervisor: Prof. Raj Kumar K

Date of Submission: 14 Feb 2026

---

1. Introduction

1.1 Purpose of Phase 2

The purpose of Phase 2 is to transform the Phase 1 problem definition into a concrete technical design.
This phase focuses on:

- Designing system architecture
- Defining detailed functional and non-functional requirements
- Designing database schema and data flow
- Implementing a working Proof of Concept (PoC)

1.2 Scope of Phase 2

Phase 2 includes:

- System architecture design
- Module-level design
- High-level database and ERD design
- Functional and non-functional requirements
- Development of a Proof of Concept
- Testing and validation strategy

---

2. System Overview

2.1 Product Perspective

SmartCloud Secrets Manager is a web-based standalone system that can integrate with multiple cloud providers such as AWS, Azure, and GCP.

The system consists of:

- Web dashboard (frontend)
- Backend API services
- Database for secret storage and logging
- AI Risk Analysis module
- Cloud SDK integrations

Deployment Environment:

- Web-based application
- Hosted on cloud infrastructure
- Accessible via browser/SDK

2.2 Major System Functions

- User authentication and authorization
- Secure secret storage (encrypted)
- Secret retrieval and update
- AI-based risk scoring
- Automated secret rotation
- Multi-cloud provider integration
- Audit logging and access logging
- Report generation

2.3 User Classes and Characteristics

1. End Users (Developers)

- Store and retrieve secrets
- View risk scores
- Trigger rotation

2. Administrators

- Manage users and projects
- Configure cloud providers
- View system logs and reports

3. AI Module (Internal System Component)

- Analyzes access logs
- Generates risk scores
- Suggests security actions

---

3. Functional Requirements

FR1: The system shall allow users to register and authenticate securely.
FR2: The system shall allow users to create and manage projects.
FR3: The system shall allow encrypted storage of secrets.
FR4: The system shall retrieve and decrypt secrets securely.
FR5: The system shall log all secret access events.
FR6: The system shall analyze secret usage patterns using AI.
FR7: The system shall generate a risk score for each secret.
FR8: The system shall automatically rotate secrets based on risk level or schedule.
FR9: The system shall integrate with multiple cloud providers.
FR10: The system shall generate audit logs and reports.

---

4. Non-Functional Requirements

4.1 Performance Requirements

- Secret retrieval response time < 2 seconds
- Risk scoring processed within 5 seconds
- System should support concurrent users (minimum 100)

4.2 Security Requirements

- Encryption for secret storage
- Role-Based Access Control (RBAC)
- Secure authentication (JWT-based)
- HTTPS communication
- Audit logging for traceability

4.3 Usability Requirements

- Clean dashboard UI
- Easy navigation between projects and secrets
- Clear display of risk level (Low / Medium / High)

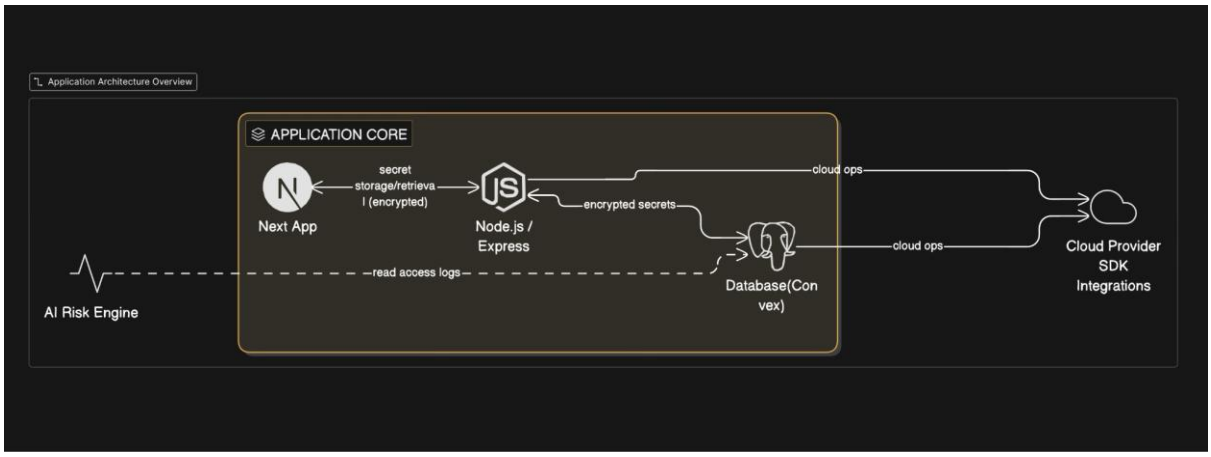
4.4 Scalability and Maintainability

- Modular backend architecture
- Microservice-ready design
- Scalable database design
- Clear separation of AI module

---

5. System Architecture and Design

5.1 System Architecture Diagram



Architecture Overview:

- Frontend (Next App)
- Backend API (Node.js / Express)
- Database (PostgreSQL) - Convex
- AI Risk Engine
- Cloud Provider SDK Integrations

5.2 Module-wise Design

1. Authentication Module

- Handles login, registration
- Issues JWT tokens
- Enforces RBAC

2. Secret Management Module

- Encrypts secrets before storage
- Retrieves and decrypts secrets
- Tracks last rotation time

3. Risk Analysis Module

- Reads access logs
- Calculates risk score
- Stores risk analysis results

4. Rotation Module

- Schedules rotation jobs
- Integrates with cloud APIs
- Updates secret after rotation

5. Logging & Reporting Module

- Stores audit logs
- Stores access logs
- Generates reports

5.3 Data Flow Design

1. User logs in
2. User stores secret → encrypted → saved in database
3. User can now use the secret directly via the SDK → access log generated
4. AI module reads access logs
5. Risk score generated and stored
6. If risk high → rotation job triggered
7. Reports generated from logs

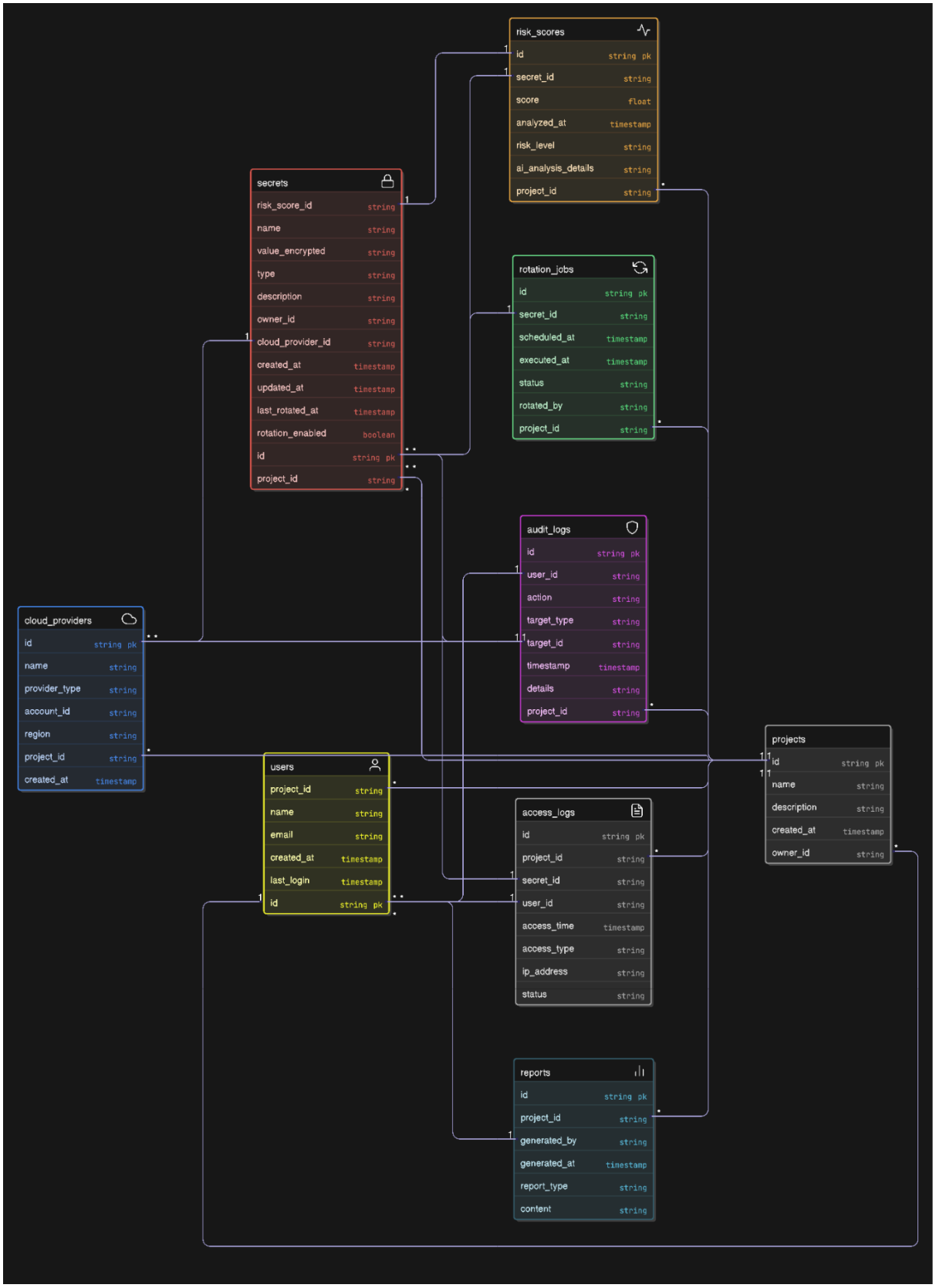
5.4 Database Design

The system uses a relational database (PostgreSQL). We will be using Convex for easy and quick implementation.

Key Entities:

- Users
- Projects
- Secrets
- Cloud Providers
- Risk Scores
- Rotation Jobs
- Access Logs
- Audit Logs
- Reports

ER Diagram


risk_scores
id string pk
secret_id string
score float
analyzed_at timestamp
risk_level string
ai_analysis_details string
project_id string
secrets
risk_score_id string
name string
value_encrypted string
type string
description string
owner_id string
cloud_provider_id string
created_at timestamp
updated_at timestamp
last_rotated_at timestamp
rotation_enabled boolean
id string pk
project_id string
rotation_jobs
id string pk
secret_id string
scheduled_at timestamp
executed_at timestamp
status string
rotated_by string
project_id string
audit_logs
id string pk
user_id string
action string
target_type string
target_id string
timestamp timestamp
details string
project_id string
cloud_providers
id string pk
name string
provider_type string
account_id string
region string
project_id string
created_at timestamp
users
project_id string
name string
email string
created_at timestamp
last_login timestamp
id string pk
projects
id string pk
name string
description string
created_at timestamp
owner_id string
access_logs
id string pk
project_id string
secret_id string
user_id string
access_time timestamp
access_type string
ip_address string
status string
reports
id string pk
project_id string
generated_by string
generated_at timestamp
report_type string
content string

Key Relationships

- One Project → Many Secrets
- One Secret → Many Risk Scores
- One Secret → Many Rotation Jobs
- One User → Many Audit Logs
- One Project → Many Access Logs

---

6. Technology Stack and Justification

Frontend

Next.js
Reason: Component-based architecture, fast UI development, large ecosystem.

Backend

Node.js + Express
Reason: Scalable API development, non-blocking I/O.

Database

PostgreSQL
Reason: Reliable relational database with strong ACID properties.

AI Module

Typescript: ***AI SDK*** by Vercel
Reason: Strong type-safety and easy implementation.

Other Tools

- JWT for authentication
- Encryption library
- Cloud SDKs (AWS SDK, Azure SDK, GCP SDK)
- GitHub for version control

---

7. Proof of Concept (PoC)

[Explanation Demo Video]

7.1 PoC Description

The PoC demonstrates:

- Secret encryption and storage

- Secret display via dashboard

Purpose: To validate secret storage and management works fine.

7.2 PoC Demonstration Details

Features Demonstrated:

- Add secret
- Retrieve/view secret via dashboard

Limitations:

- No cloud integration as of now
- Not production-optimized

---

8. Testing and Validation Strategy

Unit Testing

- Backend API endpoint testing
- Encryption/decryption validation

Integration Testing

- Secret storage + risk analysis workflow
- Rotation triggering workflow

Manual Testing

- UI interaction testing
- Role-based access testing

---

9. Risks, Challenges, and Mitigation

Identified Risks

1. Multi-cloud API Complexity

Mitigation: Abstract cloud layer using provider adapters.

2. Security Vulnerabilities

Mitigation: Code reviews, encryption testing, RBAC enforcement.

3. AI Model Accuracy

Mitigation: Start with rule-based scoring, improve iteratively.

4. Time Constraints

Mitigation: Modular development and milestone tracking.

---

10. Phase 2 Outcomes and Readiness for Phase 3

Completed in Phase 2:

- Detailed system architecture
- Database design and ERD
- Module-level design
- Defined functional & non-functional requirements
- Working PoC implementation

System Readiness:

The system is now ready for full-scale backend implementation, advanced AI integration, frontend refinement, and deployment in Phase 3.

---

11. Supervisor Review and Approval

Advisor Feedback:

Supervisor Comments: The team has demonstrated the basic working of the system architecture in the PoC which is quite reasonable.

Recommendations: Approved for submission

Signature:
Date: 14 Feb 2026